

Project Plan For Shadow AI Guardrail

How it works — core flow

Employee Sends Prompt → PII secrets stripped → Sanitised prompt → LLM → Response Returned

Project Phases:

<u>Phase 1 – Week 1-2</u> Foundation - Define PII detection rules - Set up proxy architecture - Basic portal UI (login + chat)	<u>Phase 2 – Week 3-4</u> Core Engine - Build data scrubber - Connect to LLM APIs - Audit logging system
<u>Phase 3 – Week 5-6</u> Admin Dashboard - Usage analytics per user - Policy rule editor - Alert & report exports	<u>Phase 4 – Week 7-8</u> Polish & Sell - SSO / company login - Onboarding flow - Pricing & billing page

Core features to build

- PII scrubber

* Strips names, emails, phone numbers, credit cards, internal codes before the prompt reaches the AI.

- Safe chat portal

* Simple employee-facing interface — looks like ChatGPT, but routes through your proxy.

- Audit log

* Every prompt (redacted) and response is logged with user, timestamp, and what was stripped.

- Admin dashboard

* Managers see usage reports, blocked patterns, and can set custom policies per department.

- Policy rule editor

* No-code interface to define what counts as sensitive for your company (e.g. project codenames).

Tech stack

- * Frontend (portal)
 - React.js
 - Tailwind CSS for UI
- * Backend / Proxy
 - Python
 - Express
- MS PII lib
- * AI & Detection
 - Claude API (Anthropic)
 - OpenAI (Optional)
 - Regex + NER models
- * Storage & Auth
 - Postgres for logs
 - Supabase (BaaS)
 - Auth0 / Clerk SSO

Summary Of how to approach

The core of the product is a "man-in-the-middle" proxy — employees type prompts into your portal, a scrubber strips sensitive data, then the clean prompt goes to the AI. The AI's response comes back and gets logged. The employee never needs to know the difference.

The biggest technical challenge is the PII scrubber. Microsoft's open-source **Presidio** library does the heavy lifting for standard data types (names, emails, credit cards). You'd layer custom regex rules on top for company-specific data like internal project codes.

The easiest path to a demo is:

1. A simple chat UI (React)
2. A Python backend that intercepts prompts, runs Presidio, calls the Claude API
3. A basic log table in Supabase